



# Event Wind Lookup Model

Wind-at-Point Query over a Stored Tropical-Cyclone Trajectory

MKM Research Labs

Version 1.0 — 21-July-2026

David K Kelly

**CONFIDENTIAL — PROPRIETARY**

## **Legal Notice**

### **PROPRIETARY AND CONFIDENTIAL**

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

# Contents

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>1</b> | <b>Executive Summary</b>                      | <b>4</b>  |
| <b>2</b> | <b>Model Purpose and Scope</b>                | <b>4</b>  |
| 2.1      | Purpose . . . . .                             | 4         |
| 2.2      | Scope . . . . .                               | 4         |
| 2.3      | Out of Scope . . . . .                        | 5         |
| <b>3</b> | <b>Mathematical Framework</b>                 | <b>5</b>  |
| 3.1      | Query Composition . . . . .                   | 5         |
| 3.2      | State Interpolation . . . . .                 | 5         |
| 3.3      | Clamping . . . . .                            | 6         |
| 3.4      | Geometry . . . . .                            | 6         |
| <b>4</b> | <b>Input Parameters and Calibration</b>       | <b>6</b>  |
| 4.1      | Model Inputs . . . . .                        | 6         |
| 4.2      | Inherited Parameters . . . . .                | 6         |
| <b>5</b> | <b>Implementation Details</b>                 | <b>7</b>  |
| 5.1      | Source Modules . . . . .                      | 7         |
| 5.2      | Interfaces . . . . .                          | 7         |
| 5.3      | Performance Characteristics . . . . .         | 7         |
| 5.4      | Unit Discipline . . . . .                     | 8         |
| <b>6</b> | <b>Sensitivity Analysis</b>                   | <b>8</b>  |
| <b>7</b> | <b>Validation and Backtesting</b>             | <b>10</b> |
| 7.1      | Verification Approach . . . . .               | 10        |
| 7.2      | Backtesting . . . . .                         | 11        |
| 7.3      | Automated Test Results . . . . .              | 11        |
| 7.4      | Test Document History . . . . .               | 13        |
| <b>8</b> | <b>Model Limitations and Known Weaknesses</b> | <b>13</b> |
| <b>9</b> | <b>Governance Validation Questions</b>        | <b>14</b> |
| 9.1      | Q1 — Model purpose and use . . . . .          | 14        |
| 9.2      | Q2 — Conceptual soundness . . . . .           | 14        |
| 9.3      | Q3 — Data quality . . . . .                   | 15        |
| 9.4      | Q4 — Assumptions . . . . .                    | 15        |
| 9.5      | Q5 — Implementation verification . . . . .    | 15        |
| 9.6      | Q6 — Sensitivity and stability . . . . .      | 15        |
| 9.7      | Q7 — Limitations and weaknesses . . . . .     | 15        |
| 9.8      | Q8 — Ongoing monitoring . . . . .             | 15        |
| 9.9      | Q9 — Governance and approval . . . . .        | 15        |

**10 Change History**

**15**

# 1 Executive Summary

The Event Wind Lookup Model (MKM-WS-001) answers a single question: given a stored tropical-cyclone event, what was the sustained wind speed at an arbitrary point  $(\lambda, \phi)$  at an arbitrary hour  $t$ ? It is the query layer that sits between the Tropical Cyclone Progression and Wind-Field model (MKM-TC-001), which generates and persists event trajectories, and the Wind Damage model (MKM-WD-001), which converts per-property peak winds into damage ratios and PRS event counts.

The model contributes no new physics. The radial profile, the motion-linked asymmetry correction and the surface reduction are all owned by MKM-TC-001; MKM-WS-001 composes them with two additions of its own:

- **Temporal interpolation.** Trajectories are persisted on a discrete hourly grid. A query at a non-grid hour linearly interpolates the latent state — with circular interpolation on the heading to handle the  $0/360^\circ$  seam and nearest-neighbour assignment for the discrete regime and land flags. Queries outside the trajectory’s time range clamp to the first or last stored state; the model never extrapolates.
- **Event resolution and caching.** A query accepts a path to an `EVT-*.json` file, a raw JSON payload or an already-materialised trajectory. The `WindSpeedModel` class binds a catchment configuration once and caches parsed events, so repeated lookups against the same event do not re-parse JSON.

The model is catchment-agnostic: it consumes a `WindFieldParams` bundle and a land mask, both typically drawn from the active `CatchmentTyphoonConfig`, and holds no catchment constants of its own.

**Status:** Development (Tier 3, RAG Amber). The query path, interpolation, loader and time-series helpers are delivered with unit tests. The model inherits the calibration status of MKM-TC-001: the wind-field parameters are engineering-judgement seeds, so absolute wind speeds are directional rather than calibrated.

**Materiality note:** The model is deliberately thin. Its material risk is not in its own mathematics but in the possibility of *silently* returning a plausible number for a query that should have been rejected — see Section 8, WS-L1 (clamping) and WS-L2 (no staleness check against the source event).

## 2 Model Purpose and Scope

### 2.1 Purpose

To provide a single, reusable wind-at-point query so that every downstream consumer — the per-property wind time-series stage of the port pipeline, the wind-damage composer, ad-hoc analytical queries and the Trader’s Workstation wind views — evaluates the wind field identically. Before this model existed, the evaluation order (geometry, profile, asymmetry, surface) risked being re-implemented per consumer.

### 2.2 Scope

In scope:

- Sustained wind speed in m/s at a point, for a stored event, at an hour.
- Linear interpolation of the latent typhoon state between stored hours.
- Multi-hour queries at one point (`windspeed_series`).
- Event loading from `EVT-*.json`, including tolerant stripping of the pipeline's `summary` header block.

### 2.3 Out of Scope

- Generating trajectories — owned by MKM-TC-001.
- The wind-field physics itself (radial profile, asymmetry, surface reduction) — owned by MKM-TC-001; this model calls it.
- Gust factors, gust duration statistics and directional gust conversion.
- Damage, loss or pricing — owned by MKM-WD-001 and the PRS pricer.
- Bulk per-property pre-computation. The port pipeline writes per-property wind time-series to disk; this model serves ad-hoc queries at points not in the configured portfolio, or on a finer time grid than the pipeline's  $\Delta t$ .

## 3 Mathematical Framework

### 3.1 Query Composition

For an event with time-ordered states  $\{s_i\}$ , a query at hour  $t$  and point  $x = (\lambda_x, \phi_x)$  evaluates

$$V(x, t) = \underbrace{\rho_{\text{surf}}(x)}_{\text{surface}} \underbrace{(1 + \epsilon \cos(\theta - \varphi))}_{\text{asymmetry}} \underbrace{V_{\text{sym}}(r; s(t))}_{\text{radial profile}},$$

where  $s(t)$  is the interpolated state at hour  $t$ ,  $r$  is the great-circle distance from the storm centre to  $x$  and  $\theta$  is the forward azimuth from the centre to  $x$ . The result is floored at zero.

### 3.2 State Interpolation

Let  $s_a$  and  $s_b$  be the stored states bracketing  $t$ , with  $\tau = (t - t_a)/(t_b - t_a) \in [0, 1]$ . Continuous fields — position, translation speed,  $V_{\text{max}}$ ,  $R_{\text{max}}$ ,  $R_{\text{outer}}$  and time — are interpolated linearly:

$$z(t) = z_a + \tau(z_b - z_a).$$

Heading is interpolated on the circle. Writing  $\Delta\psi$  for the signed compass difference  $\psi_b \ominus \psi_a$  taken on the shorter arc,

$$\psi(t) = (\psi_a + \tau \Delta\psi) \bmod 360^\circ,$$

which avoids the pathological interpolation a naive linear blend would produce across the  $0/360^\circ$  seam (for example  $350^\circ$  to  $10^\circ$  blending through  $180^\circ$  rather than through  $0^\circ$ ).

Discrete fields — the regime class  $k$  and the land flag — take the nearest neighbour,  $k(t) = k_a$  for  $\tau < 0.5$  and  $k_b$  otherwise. Interpolating a categorical variable is not meaningful, and a mid-point switch keeps the field piecewise constant with a single transition between stored steps.

### 3.3 Clamping

Queries outside the stored time range return the boundary state:

$$s(t) = \begin{cases} s_1 & t \leq t_1, \\ s_n & t \geq t_n, \\ \text{interpolated} & \text{otherwise.} \end{cases}$$

This is a deliberate design choice, not a numerical guard: extrapolating a cyclone trajectory beyond its simulated horizon would fabricate hazard. The consequence is that an out-of-range query returns a valid-looking number rather than an error — see WS-L1.

### 3.4 Geometry

Distance uses the Haversine great-circle formula in its `atan2` form, which is stable for closely-spaced points:

$$a = \sin^2 \frac{\Delta\phi}{2} + \cos \phi_c \cos \phi_x \sin^2 \frac{\Delta\lambda}{2}, \quad r = 2R_\oplus \operatorname{atan2}(\sqrt{a}, \sqrt{1-a}).$$

Azimuth is the standard forward-bearing formula, returned in compass degrees on  $[0, 360)$ . The bearing is undefined at  $r = 0$ ; the implementation substitutes  $\theta = 0$  there, which is harmless because the asymmetry term at the storm centre multiplies the eye-floor wind  $\alpha_{\text{eye}} V_{\text{max}}$  by a factor that is bounded within  $1 \pm \epsilon$  for any azimuth.

## 4 Input Parameters and Calibration

### 4.1 Model Inputs

Table 1: Query inputs

| Input                          | Type                                               | Source                 |
|--------------------------------|----------------------------------------------------|------------------------|
| <code>event</code>             | path / JSON / trajectory                           | MKM-TC-001 event store |
| <code>hour</code>              | float, hours since genesis                         | caller                 |
| <code>lon, lat</code>          | degrees E / N                                      | property CDM or caller |
| <code>wind_field_params</code> | WindFieldParams                                    | CatchmentTyphoonConfig |
| <code>land_mask</code>         | callable $(\lambda, \phi) \rightarrow \text{bool}$ | CatchmentTyphoonConfig |

### 4.2 Inherited Parameters

The model owns no calibrated parameters. All wind-field constants are read from the catchment configuration and are the property of MKM-TC-001; they are reproduced here because they govern the numbers in Section 6.

Table 2: WindFieldParams defaults (config/typhoon/\_field.py)

| Parameter                 | Meaning                                 | Default |
|---------------------------|-----------------------------------------|---------|
| $\alpha_{\text{eye}}$     | inner-core floor ratio                  | 0.40    |
| $p$                       | outer-decay exponent                    | 1.50    |
| $V_{\text{outer,ref}}$    | anchor wind at $R_{\text{outer}}$ (m/s) | 17.50   |
| $\epsilon_{\text{max}}$   | asymmetry cap                           | 0.30    |
| $c_{\epsilon}$            | asymmetry scaling                       | 0.60    |
| $\eta$                    | asymmetry stabiliser (m/s)              | 1.00    |
| $\varphi_{\text{offset}}$ | asymmetry phase offset (deg)            | 90.0    |
| $\rho_{\text{sea}}$       | surface factor over sea                 | 1.00    |
| $\rho_{\text{land}}$      | surface factor over land                | 0.80    |

The  $+90^\circ$  phase offset is the Northern Hemisphere convention, placing the asymmetry peak to the right of the direction of motion. A Southern Hemisphere catchment requires  $-90^\circ$ ; this is a per-catchment configuration value, not a model constant.

## 5 Implementation Details

### 5.1 Source Modules

Table 3: Implementation map

| Module                                     | Responsibility                          |
|--------------------------------------------|-----------------------------------------|
| src/models/windspeed/query.py              | windspeed(), WindSpeedModel             |
| src/models/windspeed/interpolation.py      | state interpolation and clamping        |
| src/models/windspeed/loader.py             | EVT-*.json $\rightarrow$ trajectory     |
| src/models/windspeed/timeseries.py         | multi-hour convenience wrapper          |
| <i>Called into (owned by MKM-TC-001):</i>  |                                         |
| src/models/typhoon/wind_field/point.py     | evaluation order                        |
| src/models/typhoon/wind_field/radial.py    | $V_{\text{sym}}(r)$ , decay calibration |
| src/models/typhoon/wind_field/asymmetry.py | $\epsilon$ , $\varphi$ , factor         |
| src/models/typhoon/wind_field/surface.py   | $\rho_{\text{surf}}$                    |
| src/models/typhoon/wind_field/geometry.py  | Haversine, bearing                      |

### 5.2 Interfaces

Two entry points are offered deliberately. The free function `windspeed(event, hour, lon, lat, *, wind_field_params, land_mask)` is stateless and suits one-shot queries. The `WindSpeedModel` class binds a `CatchmentTyphoonConfig` at construction and caches loaded events by source key; it is the correct choice inside any loop over properties or hours. The class is callable, so `ws(event, hour, lon, lat)` and `ws.query(...)` are equivalent.

### 5.3 Performance Characteristics

Interpolation performs a linear scan over the state list. For the 168-state hourly trajectories the pipeline produces this is adequate and keeps the code obvious; a bisection search becomes worthwhile



only if event horizons grow by an order of magnitude. The dominant cost in a portfolio-scale sweep is JSON parsing, which the model's event cache removes for all queries after the first against a given event.

## 5.4 Unit Discipline

Wind speeds are m/s throughout the model's own surface. The latent state stores translation speed in km/h, and the conversion to m/s happens inside the asymmetry term where the two quantities must be commensurate. Callers should treat every value returned by this model as m/s; conversion to km/h, if needed for presentation, belongs in the presentation layer.

## 6 Sensitivity Analysis

The tables below are generated by calling the model directly (`python -m docs.models.sensitivities.generate --model windspeed`). They demonstrate the four axes along which the query responds: radial distance, storm intensity, azimuth relative to motion, and surface class — plus the temporal interpolation behaviour.

Three features are worth reading closely:

- The radial profile is *not* monotone in  $V_{\max}$  at long range. Beyond roughly 250 km a more intense storm produces *lower* wind than a weaker one, because the outer profile is anchored at  $V_{\text{outer,ref}}$  at a fixed  $R_{\text{outer}}$ : a higher peak wind must decay faster to meet the same anchor. This is a property of the anchored parameterisation, not a defect, but it means the model should not be used to rank storm severity at points far outside the gale radius.
- The decay-length calibration is ill-conditioned just above the anchor wind. At  $V_{\max} = 20$  m/s the solved  $L$  exceeds 800 km, against 213 km at  $V_{\max} = 50$  m/s, because  $-\ln(V_{\text{ref}}/V_{\max})$  approaches zero. See WS-L3.
- Asymmetry is modest at realistic translation speeds: at 18 km/h the multiplier spans roughly 0.96 to 1.06. It becomes material only for fast-moving systems.

Table 4: Symmetric radial profile  $V_{sym}(r)$  in m/s by distance from the storm centre ( $R_{max} = 30$  km,  $R_{outer} = 250$  km).

| $r$ (km) | $V_{sym}$ at $V_{max} = 35$ | $V_{sym}$ at $V_{max} = 50$ | $V_{sym}$ at $V_{max} = 65$ |
|----------|-----------------------------|-----------------------------|-----------------------------|
| 0        | 14.0                        | 20.0                        | 26.0                        |
| 10       | 21.0                        | 30.0                        | 39.0                        |
| 20       | 28.0                        | 40.0                        | 52.0                        |
| 30       | 35.0                        | 50.0                        | 65.0                        |
| 50       | 34.3                        | 48.6                        | 62.7                        |
| 75       | 32.8                        | 45.4                        | 57.6                        |
| 100      | 30.9                        | 41.4                        | 51.4                        |
| 150      | 26.5                        | 32.8                        | 38.3                        |
| 200      | 21.9                        | 24.5                        | 26.7                        |
| 250      | 17.5                        | 17.5                        | 17.5                        |
| 300      | 13.6                        | 12.0                        | 10.9                        |
| 400      | 7.72                        | 5.06                        | 3.72                        |

Table 5: Outer-decay length  $L$  (km) by peak wind, anchored at  $V_{outer,ref} = 17.5$  m/s with  $p = 1.5$ . The geometric fallback applies when the eyewall wind is at or below the anchor.

| $V_{max}$ (m/s) | $L$ (km) | Fallback |
|-----------------|----------|----------|
| 15.0            | 220.0    | yes      |
| 17.5            | 220.0    | yes      |
| 20.0            | 842.1    | no       |
| 30.0            | 332.2    | no       |
| 40.0            | 249.8    | no       |
| 50.0            | 213.0    | no       |
| 60.0            | 191.4    | no       |
| 70.0            | 176.9    | no       |

Table 6: Asymmetry multiplier  $1 + \epsilon \cos(\theta - \phi)$  by translation speed and azimuth ( $V_{max} = 50$  m/s, heading  $315^\circ$ ,  $\phi = 45^\circ$ ,  $\epsilon_{max} = 0.3$ ).

| $u$ (km/h) | $\epsilon$ | $\theta = 0^\circ$ | $\theta = 45^\circ$ | $\theta = 90^\circ$ | $\theta = 135^\circ$ | $\theta = 180^\circ$ | $\theta = 270^\circ$ |
|------------|------------|--------------------|---------------------|---------------------|----------------------|----------------------|----------------------|
| 0.0000     | 0.0000     | 1.00               | 1.00                | 1.00                | 1.00                 | 1.00                 | 1.00                 |
| 5.00       | 0.0163     | 1.01               | 1.02                | 1.01                | 1.00                 | 0.9884               | 0.9884               |
| 10.0       | 0.0327     | 1.02               | 1.03                | 1.02                | 1.00                 | 0.9769               | 0.9769               |
| 18.0       | 0.0588     | 1.04               | 1.06                | 1.04                | 1.00                 | 0.9584               | 0.9584               |
| 25.0       | 0.0817     | 1.06               | 1.08                | 1.06                | 1.00                 | 0.9422               | 0.9422               |
| 35.0       | 0.1144     | 1.08               | 1.11                | 1.08                | 1.00                 | 0.9191               | 0.9191               |
| 50.0       | 0.1634     | 1.12               | 1.16                | 1.12                | 1.00                 | 0.8845               | 0.8845               |

Table 7: End-to-end point wind (m/s) due north of the storm centre, over sea vs over land ( $\rho_{sea} = 1.0$ ,  $\rho_{land} = 0.8$ ).

| Distance (km) | Sea (m/s) | Land (m/s) | Difference |
|---------------|-----------|------------|------------|
| 0             | 20.8      | 16.7       | -4.17      |
| 25            | 46.9      | 37.5       | -9.37      |
| 50            | 50.6      | 40.5       | -10.1      |
| 100           | 43.1      | 34.5       | -8.63      |
| 150           | 34.1      | 27.3       | -6.82      |
| 250           | 18.2      | 14.6       | -3.65      |

Table 8: Linear state interpolation between stored hours 12 and 18. Queries outside the trajectory range clamp to the first / last stored state.

| Hour | $V_{max}$ (m/s) | $R_{max}$ (km) | Clamped |
|------|-----------------|----------------|---------|
| 6.00 | 40.0            | 40.0           | yes     |
| 12.0 | 40.0            | 40.0           | no      |
| 13.5 | 45.0            | 36.2           | no      |
| 15.0 | 50.0            | 32.5           | no      |
| 16.5 | 55.0            | 28.8           | no      |
| 18.0 | 60.0            | 25.0           | no      |
| 24.0 | 60.0            | 25.0           | yes     |

## 7 Validation and Backtesting

### 7.1 Verification Approach

The model is verified by unit test against properties that hold independently of calibration:

- **Interpolation exactness.** A query at a stored hour returns the stored state unchanged.
- **Interpolation monotonicity.** For a monotone field, the interpolated value at the midpoint lies between the bracketing values.
- **Circular heading.** Interpolating  $350^\circ \rightarrow 10^\circ$  crosses  $0^\circ$ , not  $180^\circ$ .
- **Clamping.** Queries before the first and after the last stored hour return the boundary states exactly.
- **Loader tolerance.** An EVT-\*.json payload carrying the pipeline’s `summary` header parses identically to one without it; a missing file raises `FileNotFoundError`.
- **Cache transparency.** A cached query returns the same value as an uncached one, and `clear_cache()` restores the uncached path.
- **Series consistency.** `windspeed_series` over  $n$  hours equals  $n$  independent `windspeed` calls.

## 7.2 Backtesting

No backtest against observed anemometer records has been performed. Such a backtest would validate MKM-TC-001's wind-field parameterisation rather than this model's composition logic, and is tracked as a remediation item against that model. This model's own correctness claim rests on the verification properties above.

## 7.3 Automated Test Results

Table 9: Test Results Summary — Event Wind Lookup (MKM-WS-001)

|             |       |
|-------------|-------|
| Total Tests | 56    |
| Passed      | 56    |
| Failed      | 0     |
| Skipped     | 0     |
| Duration    | 0.02s |

Table 10: Detailed Test Results — Event Wind Lookup

| Test Description                    | Acceptance Criteria                                                           | Result | Time   |
|-------------------------------------|-------------------------------------------------------------------------------|--------|--------|
| After last returns last             | <code>s == simple_trajectory[-1]</code>                                       | Pass   | 0.000s |
| After last returns last             | <code>s == simple_trajectory[-1]</code>                                       | Pass   | 0.000s |
| At first endpoint returns first     | <code>s == simple_trajectory[0]</code>                                        | Pass   | 0.000s |
| At first endpoint returns first     | <code>s == simple_trajectory[0]</code>                                        | Pass   | 0.000s |
| At second endpoint returns second   | <code>s == simple_trajectory[-1]</code>                                       | Pass   | 0.000s |
| At second endpoint returns second   | <code>s == simple_trajectory[-1]</code>                                       | Pass   | 0.000s |
| Before first returns first          | <code>s == simple_trajectory[0]</code>                                        | Pass   | 0.000s |
| Before first returns first          | <code>s == simple_trajectory[0]</code>                                        | Pass   | 0.000s |
| Duplicate timestamps return earlier | <code>s.longitude == 0.0</code>                                               | Pass   | 0.000s |
| Duplicate timestamps return earlier | <code>s.longitude == 0.0</code>                                               | Pass   | 0.000s |
| Empty states raises                 | —                                                                             | Pass   | 0.000s |
| Empty states raises                 | —                                                                             | Pass   | 0.000s |
| Heading circular short arc          | <code>s.heading_deg == pytest.approx(0.0, abs=1e-6)</code>                    | Pass   | 0.000s |
| Heading circular short arc          | <code>s.heading_deg == pytest.approx(0.0, abs=1e-6)</code>                    | Pass   | 0.000s |
| Land flag takes nearest neighbour   | <code>early.land_flag is False;</code><br><code>late.land_flag is True</code> | Pass   | 0.000s |
| Land flag takes nearest neighbour   | <code>early.land_flag is False;</code><br><code>late.land_flag is True</code> | Pass   | 0.000s |

| Test Description                 | Acceptance Criteria                                                                                                              | Result | Time   |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|--------|--------|
| Midpoint lerps continuous fields | <code>s.longitude == pytest.approx(5.25, abs=1e-9);</code><br><code>s.v_max.ms == pytest.approx(40.0, abs=1e-9)</code> (+1 more) | Pass   | 0.000s |
| Midpoint lerps continuous fields | <code>s.longitude == pytest.approx(5.25, abs=1e-9);</code><br><code>s.v_max.ms == pytest.approx(40.0, abs=1e-9)</code> (+1 more) | Pass   | 0.000s |
| Regime takes nearest neighbour   | <code>early.regime is RegimeClass.STRAIGHT_WESTWARD;</code><br><code>late.regime is RegimeClass.LANDFALL_DECAY</code>            | Pass   | 0.000s |
| Regime takes nearest neighbour   | <code>early.regime is RegimeClass.STRAIGHT_WESTWARD;</code><br><code>late.regime is RegimeClass.LANDFALL_DECAY</code>            | Pass   | 0.000s |
| Accepts string path              | <code>restored.event_id == traj.event_id</code>                                                                                  | Pass   | 0.001s |
| Accepts string path              | <code>restored.event_id == traj.event_id</code>                                                                                  | Pass   | 0.001s |
| Missing file raises              | —                                                                                                                                | Pass   | 0.000s |
| Missing file raises              | —                                                                                                                                | Pass   | 0.000s |
| Passes trajectory through        | <code>load_event(traj) is traj</code>                                                                                            | Pass   | 0.000s |
| Passes trajectory through        | <code>load_event(traj) is traj</code>                                                                                            | Pass   | 0.000s |
| Round trip via pipeline writer   | <code>restored.event_id == traj.event_id;</code><br><code>restored.scenario_family is ScenarioFamily.BASELINE</code> (+1 more)   | Pass   | 0.002s |
| Round trip via pipeline writer   | <code>restored.event_id == traj.event_id;</code><br><code>restored.scenario_family is ScenarioFamily.BASELINE</code> (+1 more)   | Pass   | 0.001s |
| Strips summary header            | <code>"summary" in payload;</code><br><code>restored.event_id == traj.event_id</code>                                            | Pass   | 0.001s |
| Strips summary header            | <code>"summary" in payload;</code><br><code>restored.event_id == traj.event_id</code>                                            | Pass   | 0.001s |
| Caches loaded events by path     | <code>ws.cached_event_keys == (); str(path)</code><br><code>in ws.cached_event_keys</code> (+1 more)                             | Pass   | 0.002s |
| Caches loaded events by path     | <code>ws.cached_event_keys == (); str(path)</code><br><code>in ws.cached_event_keys</code> (+1 more)                             | Pass   | 0.001s |
| Call is query alias              | <code>ws(traj, 1.0, 5.0, 5.0) == ws.query(traj, 1.0, 5.0, 5.0)</code>                                                            | Pass   | 0.000s |
| Call is query alias              | <code>ws(traj, 1.0, 5.0, 5.0) == ws.query(traj, 1.0, 5.0, 5.0)</code>                                                            | Pass   | 0.000s |

| Test Description                      | Acceptance Criteria                                                                | Result | Time   |
|---------------------------------------|------------------------------------------------------------------------------------|--------|--------|
| Clear cache                           | <code>ws.cached_event_keys != ()</code><br><code>ws.cached_event_keys == ()</code> | Pass   | 0.002s |
| Clear cache                           | <code>ws.cached_event_keys != ()</code><br><code>ws.cached_event_keys == ()</code> | Pass   | 0.001s |
| In memory trajectory is not cached    | <code>ws.cached_event_keys == ()</code>                                            | Pass   | 0.000s |
| In memory trajectory is not cached    | <code>ws.cached_event_keys == ()</code>                                            | Pass   | 0.000s |
| Query matches function form           | <code>a == pytest.approx(b, rel=1e-9)</code>                                       | Pass   | 0.000s |
| Query matches function form           | <code>a == pytest.approx(b, rel=1e-9)</code>                                       | Pass   | 0.000s |
| Accepts path via pipeline writer      | <code>v_from_path ==</code><br><code>pytest.approx(v_from_obj, rel=1e-9)</code>    | Pass   | 0.002s |
| Accepts path via pipeline writer      | <code>v_from_path ==</code><br><code>pytest.approx(v_from_obj, rel=1e-9)</code>    | Pass   | 0.002s |
| Far point has lower wind              | <code>v_far &lt; v_near</code>                                                     | Pass   | 0.000s |
| Far point has lower wind              | <code>v_far &lt; v_near</code>                                                     | Pass   | 0.000s |
| Returns float at query point          | <code>isinstance(v, float); v &gt;= 0.0</code>                                     | Pass   | 0.000s |
| Returns float at query point          | <code>isinstance(v, float); v &gt;= 0.0</code>                                     | Pass   | 0.000s |
| Strength grows as storm intensifies   | <code>v_late &gt; v_early</code>                                                   | Pass   | 0.000s |
| Strength grows as storm intensifies   | <code>v_late &gt; v_early</code>                                                   | Pass   | 0.000s |
| Empty hours returns empty             | <code>series == []</code>                                                          | Pass   | 0.000s |
| Empty hours returns empty             | <code>series == []</code>                                                          | Pass   | 0.000s |
| Matches pointwise windspeed           | <code>v == pytest.approx(v_single,</code><br><code>rel=1e-12)</code>               | Pass   | 0.000s |
| Matches pointwise windspeed           | <code>v == pytest.approx(v_single,</code><br><code>rel=1e-12)</code>               | Pass   | 0.000s |
| Peaks when storm centre passes nearby | <code>hours[peak_idx] == 2.0</code>                                                | Pass   | 0.000s |
| Peaks when storm centre passes nearby | <code>hours[peak_idx] == 2.0</code>                                                | Pass   | 0.000s |
| Returns one value per hour            | <code>len(series) == len(hours)</code>                                             | Pass   | 0.000s |
| Returns one value per hour            | <code>len(series) == len(hours)</code>                                             | Pass   | 0.000s |

## 7.4 Test Document History

| Date        | Version | Description                   | Author   |
|-------------|---------|-------------------------------|----------|
| 21-Jul-2026 | 1.0     | Initial test suite (56 tests) | D. Kelly |

## 8 Model Limitations and Known Weaknesses

**WS-L1 — Silent clamping.** Out-of-range hours return the boundary state rather than raising. A caller that mis-computes an hour offset receives a plausible wind speed with no signal that the

query was invalid. *Mitigation:* consumers that compute hours arithmetically should assert the hour lies within the event horizon before querying.

**WS-L2 — No event-staleness check.** The loader does not verify that a cached or supplied trajectory corresponds to the event the caller believes it is querying, nor that the file has not changed on disk since it was cached. Long-lived `WindSpeedModel` instances spanning a regeneration of the event store will serve stale winds. *Mitigation:* construct the model per pipeline run; call `clear_cache()` after any event regeneration.

**WS-L3 — Ill-conditioned decay calibration near the anchor.** For  $V_{\max}$  marginally above  $V_{\text{outer,ref}}$  the solved decay length diverges, producing an implausibly flat outer profile. Storms at or below the anchor fall back to a geometric length and are explicitly not physically realistic. Sub-gale systems carry negligible damage weight, so the practical impact is small, but the model should not be used for tropical-depression-strength analysis.

**WS-L4 — Binary land mask.** Surface reduction is a two-valued function of land/sea at the evaluation point. Terrain roughness, exposure class and topographic speed-up are not represented, so winds at exposed coastal or elevated sites are understated relative to sheltered inland sites at the same distance.

**WS-L5 — Linear state interpolation.** A cyclone's intensity can change faster than linearly between stored hours, particularly during rapid intensification or at landfall. Sub-hourly queries during such phases carry more error than the hourly grid suggests.

**WS-L6 — Inherited calibration status.** Absolute wind speeds are only as good as MKM-TC-001's seed parameters. Relative comparisons (across distance, azimuth or surface class) are more reliable than absolute levels.

## 9 Governance Validation Questions

The following nine questions are mandated by Chapter 5 of the *Model Risk Governance for Vendors* handbook and must be explicitly addressed for every model in the inventory. Response status is tracked in the Model Governance Dashboard (MRC portal).

### 9.1 Q1 — Model purpose and use

*“A wind-at-point query layer over stored tropical-cyclone events. Consumed by the per-property wind time-series stage, by MKM-WD-001 for damage, and by ad-hoc analysis. It does not itself price or generate hazard.”*

### 9.2 Q2 — Conceptual soundness

*“The model composes an established parameterisation owned upstream. Its own contributions — linear state interpolation with circular heading handling, boundary clamping, and event caching — are standard and are each covered by a verification property in Section 7.”*

### 9.3 Q3 — Data quality

*“Inputs are the event store written by MKM-TC-001 and the property coordinates from the CDM. The loader tolerates the pipeline’s optional `summary` block but performs no schema validation beyond what `TyphoonTrajectory.from_dict` enforces.”*

### 9.4 Q4 — Assumptions

*“Three material assumptions: that the latent state varies linearly between stored hours (WS-L5); that clamping rather than extrapolation is the correct treatment outside the event horizon; and that a binary land/sea surface factor is adequate (WS-L4).”*

### 9.5 Q5 — Implementation verification

*“Unit tests cover the query function, the model class and its cache, interpolation including the circular-heading and clamping edge cases, the loader including the missing-file and summary-block paths, and the series wrapper.”*

### 9.6 Q6 — Sensitivity and stability

*“See Section 6. The response is smooth and monotone in distance within the gale radius and in azimuth. The non-monotonicity in  $V_{\max}$  at long range and the ill-conditioned decay calibration near the anchor wind are documented as WS-L3 and are properties of the upstream parameterisation.”*

### 9.7 Q7 — Limitations and weaknesses

*“Documented in Section 8 (WS-L1–WS-L6). Silent clamping (WS-L1) and event staleness (WS-L2) are the operationally material ones because both fail quietly.”*

### 9.8 Q8 — Ongoing monitoring

*“Not yet in production. The monitoring plan follows MKM-TC-001: distribution checks on queried peak winds per event, and an alert on any query whose hour is clamped.”*

### 9.9 Q9 — Governance and approval

*“Owner: David K Kelly. Status: Development, Tier 3, RAG Amber. Approval is contingent on MKM-TC-001 calibration, since this model inherits it.”*

## 10 Change History

| Version | Date         | Description                                                                        |
|---------|--------------|------------------------------------------------------------------------------------|
| 1.0     | 21-July-2026 | Initial model documentation: framework, inherited parameter set, implementation ma |